Information Security
Quiz Refresher: Chapter 2, Questions

The web and stateful tracking: the cookie attachment rule and its attributes, sites and origins, cross-site tracking, respawning, cookie syncing and RTB, clickjacking; answers in the separate answer sheet

Information Security

2026-10-29

How to use this document

Three rounds, answers in the separate answer sheet. Round A is 24 one-line recall questions. Round B is eight statement sets in the course quiz's format: four statements, mark every true one (zero to four) and justify each mark in one line. Round C is six exam-style problems; every host name, cookie and number is new, so the rules must carry you, not the chapter's worked quiz. Budget 25 minutes for A and B, 60 minutes for C.

Round A: rapid recall

A1. The course's definition of privacy has four data-life-cycle verbs and three agency verbs. Name them. Which agency verb does *invisible* tracking defeat, and which does tracking that survives cookie deletion defeat?

A2. Name the five parts of this URL:
`https://informationsecurity.com:443/courses/web-security?week=5&topic=xss`

A3. What does "HTTP is stateless" mean exactly? Why does that force every web application to add state, and why does that matter for tracking?

A4. "A script runs with the authority of . . . ". Complete the sentence and state what it implies for `analytics.js` served by `metrics.test` and included by `shop.test`.

A5. Is the URL fragment (`#...`) sent in the HTTP request? How can a third party still learn it?

A6. Session identifier in the URL: name three ways it leaks.

A7. State the cookie attachment rule with both of its clauses. Which clause does `SameSite` qualify?

A8. What makes a cookie a session cookie rather than a persistent one? How does a server end a session?

A9. `Domain` attribute: the default scope, what you may widen it to, and what you may not set it to.

A10. `HttpOnly`: what does it block, and name two things it does *not* prevent.

A11. `Secure`: its two effects.

A12. Define `SameSite=Strict`, `Lax` and `None`. Give Lax's two conditions and None's requirement. What is the default when the attribute is absent?

A13. Give the four-step procedure for deciding whether cookie c is attached to a request from page P to destination D. Which attribute never enters it?

A14. Define eTLD, registrable domain, first-party and third-party request.

A15. Define an origin. What does the same-origin policy govern, and what does it never prevent? Are two subdomains same-site, same-origin, both, or neither?

A16. Name the three capabilities a tracker needs and what is lost when each is removed.

A17. State the "one third-party cookie" theorem and say which clause of the attachment rule each step of the proof uses.

A18. Why can first-party analytics measure repeat visits within a site but not link visits across sites? How does the analytics script get its identifier to the vendor?

A19. Does "block third-party cookies" stop first-party analytics? Name the loophole.

A20. Define respawning. Name three stores it can use, each with its scope and the clearing action that empties it.

A21. Explain ETag tracking in three steps. Which lawsuit is the course's example, and what did the trace show?

A22. Define cookie syncing. In which part of which message does the identifier cross the domain boundary, and why can no script do the same?

A23. Real-time bidding: the roles of the SSP, the ad exchange and a DSP. Who receives the bid request, and what does it contain?

A24. Clickjacking relies on three browser facts; name them. Name the two framing-control headers, who must send them, and which one can whitelist a specific third origin.

Round B: statement sets, mark every true statement

Each set is 4 points: one per statement, letter plus a one-line justification.

B1. **A bank cookie.** `https://bank.test` sets `sid=9; Secure; HttpOnly` with no `SameSite` attribute, in a current browser.
(A) A script injected into a bank page can read `sid` through `document.cookie`.
(B) Clicking a link in `https://mail.test` to `https://bank.test/account` sends `sid`.
(C) `https://evil.test` auto-submits a POST form to `https://bank.test/transfer`; `sid` is sent.
(D) `<img src="https://bank.test/logo.png">` on `evil.test` sends `sid`.

B2. **Origin and site.** The page is `https://app.example.com` (port 443).
(A) `https://api.example.com` is same-origin with it.
(B) `https://api.example.com` is same-site with it.
(C) `http://app.example.com` is same-origin with it.
(D) A cookie set by `app.example.com` with `Domain=example.com` is sent to `api.example.com`.

B3. **A tracking pixel.** `https://px.adco.test/p.gif` is embedded on `news.test` and on `shop.test`; its cookie is `u=77; SameSite=None; Secure`; third-party cookies are allowed.
(A) Both embedding sites' requests for the pixel carry `u=77`.
(B) A script on `news.test` can read `u` through `document.cookie`.
(C) `px.adco.test` learns which page embedded it from the `Referer` header.
(D) With `SameSite=Lax` instead, the pixel requests would still carry `u`.

B4. **An analytics script.** `https://stats.test/a.js` is included by `news.test` and sets a cookie `v=N1`.
(A) `v` is a first-party cookie of `news.test`.
(B) `v` is attached automatically to the request that fetches `a.js` from `stats.test`.
(C) Included on `shop.test`, the same script finds and reuses `v=N1`.
(D) The script can read `v` and send it to `stats.test` as a URL parameter.

B5. **Respawning.**
(A) "Clear cookies" also empties `localStorage`.
(B) `localStorage` is per origin, so a `tracker.test` iframe reaches the same store on every site that embeds it.
(C) An identifier stored in an `ETag` survives cookie deletion but not clearing the cache.
(D) Flash local shared objects were removed by "clear cookies".

B6. **Cookie syncing.** Tracker 1 answers its pixel request with a redirect:
`302 Location: https://tracker2.test/sync?t1=1234`
(A) Tracker 2 learns `1234` because tracker 1's cookie is attached to the redirected request.
(B) The browser attaches tracker 2's own cookie to the redirected request.
(C) A script on the publisher page could achieve the same by reading both cookies.
(D) After the sync, each tracker's coverage becomes the union of the sites both are embedded on.

B7. **Real-time bidding.**
(A) Only the DSP that wins the auction receives the bid request.
(B) The SSP acts on behalf of the publisher.
(C) The pseudonymous identifier in the bid request prevents a DSP from recognising a user it has profiled.
(D) What each DSP retains can be observed from the browser.

B8. **Clickjacking.** Eve's `game.html` frames the Student Portal with `opacity: 0`.
(A) Adding `X-Frame-Options: DENY` to `game.html`'s response protects the portal.
(B) `Content-Security-Policy: frame-ancestors 'self' https://apps.um6p.ma` on the portal allows framing by the portal itself and by `apps.um6p.ma` only.
(C) With `visibility: hidden` instead of `opacity: 0`, Alice's click would still reach the portal's button.
(D) `SameSite=Strict` on the portal's session cookie makes the framed portal load logged out.

Round C: exam-style problems

C1. The attachment rule, seven requests [10 pts]

`https://www.uni.test` sets `tok=Q1; Domain=uni.test; Path=/app; Secure; SameSite=Lax`. For each request say whether `tok` is attached and which step of the four-step procedure decides it.

(a) From `https://www.uni.test/app/home`, a script fetches `https://api.uni.test/app/data`.

(b) From `https://www.uni.test/app/home`, the user clicks a link to `https://www.uni.test/docs`.

(c) From `https://forum.test`, the user clicks a link to `https://www.uni.test/app/grades`.

(d) A page on `https://forum.test` auto-submits a POST form to `https://www.uni.test/app/submit`.

(e) `https://forum.test` embeds `<iframe src="https://www.uni.test/app/widget">`.

(f) The user types `http://www.uni.test/app/home` (plain HTTP).

(g) A page requests `https://uni.test.evil.test/app/x`.

Then: (h) which answers change if the cookie is reissued with `SameSite=None; Secure`? (i) With `SameSite=Strict`?

C2. Parties, sites and origins [8 pts]

The address bar shows `https://mail.campus.ac.uk`. The public suffix list contains `ac.uk`.

(a) Give the eTLD and the registrable domain of the visited site.

(b) Classify each destination as first or third party, with the registrable domain that decides it: `static.campus.ac.uk`; `campus.ac.uk.cdn.fastnet.test`; `campus-ac.uk`; `login.campus.ac.uk:8443`; `campus.co.uk`.

(c) Can a script running on `mail.campus.ac.uk` read `document.cookie` of `login.campus.ac.uk`? Can a cookie set with `Domain=campus.ac.uk` be sent to both? Name the rule behind each answer.

(d) May `mail.campus.ac.uk` set a cookie with `Domain=ac.uk`? Why would allowing it be dangerous?

(e) In the TP, why are `lab.test:8000` and `127.0.0.1:8000` two different cookie jars for one Flask server?

C3. Two trackers, one morning [12 pts]

A browser starts with an empty cookie jar and visits:

- 09:00 `https://news.test/politics`: the page embeds `<img src="https://px.adco.test/p.gif">` and `<script src="https://stats.test/a.js">`.
- 09:30 `https://shop.test/cart`: the same pixel and the same script.
- 10:00 `https://blog.test/post`: only the script.

The pixel server sets `u=<new id> ; SameSite=None ; Secure` when no cookie arrives. The script creates `v=<fresh id>` on the page's domain when none exists and, on every page, sends `https://stats.test/t?v=<id>&site=<hostname>`. Third-party cookies are allowed; use `u=1` and `v=a`, `v=b`, `v=c` for successive ids.

(a) Write the cookie jar after 10:00 (one line per domain).

(b) Write `px.adco.test`'s log and say what it concludes about this browser. Which header told it the site?

(c) Write `stats.test`'s log and say what it can and cannot conclude.

(d) The user logs in with the same e-mail on `news.test` and `shop.test`, and both sites pass the e-mail to the script. What changes?

(e) Now the browser blocks third-party cookies. Rewrite both logs. Which tracker is hurt, and why is the other untouched?

(f) Minimal change that turns `stats.test` into a cross-site tracker (one sentence).

C4. Reading a respawning experiment [10 pts]

A tracker page is visited six times; before each visit the tester performs an action and afterwards records the cookie (C), the `localStorage` value (L), the `ETag` echoed in `If-None-Match` (E), the fingerprint hash (F) and the profile number on the tracker's dashboard (P).

Test	Action before the visit	C	L	E	F	P
1	nothing (first visit)	4411	4411	4411	17	6
2	delete the cookie	4411	4411	4411	17	6
3	delete the cookie, clear site data (storage)	4411	4411	4411	17	6
4	clear cookies, site data and cache	8820	8820	8820	17	6
5	open a private window, same browser	9931	9931	9931	17	6
6	a different browser, same machine	1207	1207	1207	29	11

(a) For tests 2, 3 and 4, say which mechanism the result is *consistent with*, and why it is not *proved*.

(b) In test 3 the storage was cleared, yet L shows 4411 again. Explain the order of events.

(c) In test 4 every identifier changed but P is still 6. What is the only remaining explanation? Why does "consistent with" still apply?

(d) Test 5 against test 6: what do they establish about F, and what false positive can F produce?

(e) Give the single sufficient action against both stateful respawns seen here, and say why "delete cookies" and "clear site data" each fail alone.

C5. Write the sync [10 pts]

`ad-x.test` holds cookie `x=X9` for this browser and `ad-y.test` holds `y=Y4`. A page on `https://news.test` embeds a pixel from `ad-x.test`.

(a) Write the HTTP exchange of a sync initiated by `ad-x.test`: each request with its `Cookie` and `Referer` headers, each response with its status and `Location`; include the optional return step.

(b) What does each tracker know about the browser before the sync and after it?

(c) Why a redirect, rather than a script on `news.test` that reads both cookies and posts them?

(d) Which clause of the attachment rule makes step 3 carry `y=Y4`, and in which part of which message does `X9` cross the domain boundary?

(e) Firefox gives every top-level site its own cookie jar. Does the sync above still work on `news.test`? What does the partition prevent instead?

(f) Inside RTB, between which two parties does this exact exchange routinely occur, and what does the DSP gain from it?

C6. Framing the portal [10 pts]

The Student Portal at `https://portal.uni.test` is logged in. Eve's page frames it transparently over a game and aligns the portal's "Confirm" button under the game's button.

(a) List the three browser behaviours the attack relies on, and for each say what would break it.

(b) Write the response header(s) the portal should send so that nobody may frame it, including old browsers.

(c) The university later needs `https://lms.uni.test` to embed the portal. Write the header that allows exactly that, and explain why `X-Frame-Options` cannot express it.

(d) Eve argues that `SameSite=Lax` on the portal's session cookie already defeats her. Is she right about the TP attack? Why is it still not the recommended defence?

(e) Why is JavaScript frame-busting considered weak compared with the headers?